Pontificia Universidad JAVERIANA Colombia

Human-in-the-loop

Semana 9 · Persistencia, aprobación y reanudación
Tópicos Avanzados en Analítica II – IA Generativa
Maestría en Analítica para la Inteligencia de Negocios

Sergio Alberto Mora Pardo - H1 2027
GitHub · LinkedIn · Repositorio del curso

Tópicos Avanzados en Analítica II

Estructura del curso

Fundamentos con LangChain
Semanas 1–3 ✓

RAG y evaluación
Semanas 4–6 ✓

Model Context Protocol
Semana 7 ✓

LangGraph y patrones agénticos
Semanas 8–11

LLMOps y despliegue
Semana 12

Semana 9 · Human-in-the-loop

- Cuando una acción mueve dinero
- Pausar un grafo con `interrupt`
- Lo que guarda el checkpointer
- Reanudar: aprobar, rechazar o editar
- `HumanInTheLoopMiddleware` en un agente
- Por qué el control va en el código

Las ideas que traemos de la semana 8

Repaso

Antes de avanzar

1. Un grafo combina pasos fijos en código con decisiones del modelo.
2. Las aristas condicionales deciden el siguiente nodo.
3. La semana 5 nos dio el checkpointer y el `thread_id`.

Una acción que no se deshace

El problema de la semana

Leer frente a actuar

Hasta ahora los agentes solo **leían**: cartera, políticas, TRM.

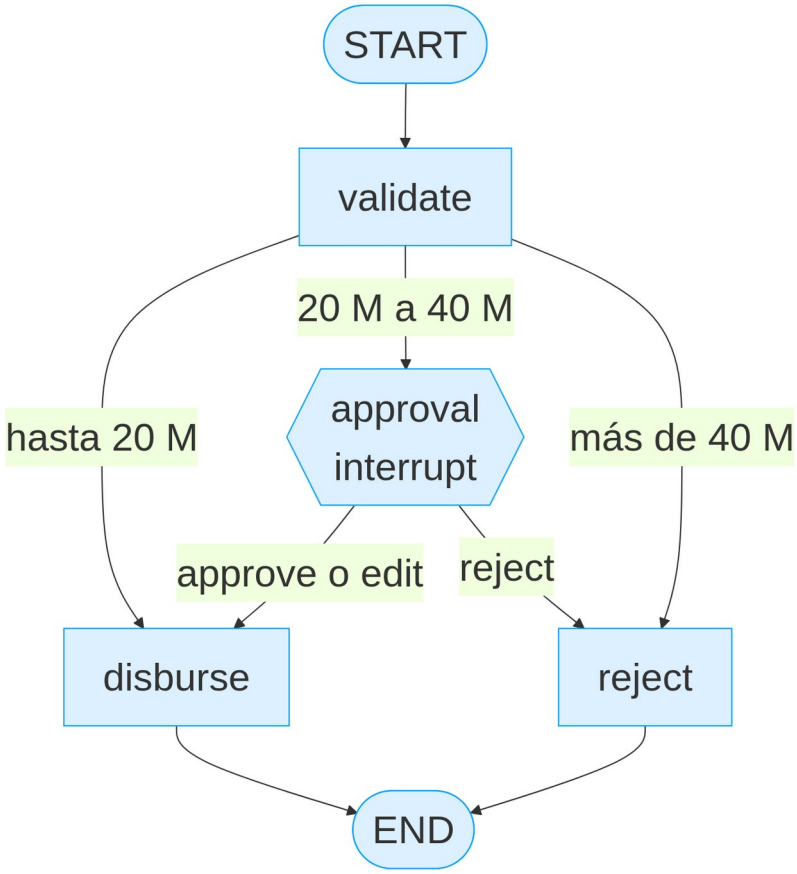
Un **desembolso** mueve dinero. La política de ACME: máximo **40.000.000 COP** por cliente al día, y un analista revisa los desembolsos grandes.

El flujo del desembolso

L20 · secciones 1 y 2

Tres caminos

Hasta 20 M: automático.
20 M a 40 M: analista.
Más de 40 M: rechazo.
Los límites están
en el código.



Pausar con interrupt

L20 · sección 2

El nodo de aprobación

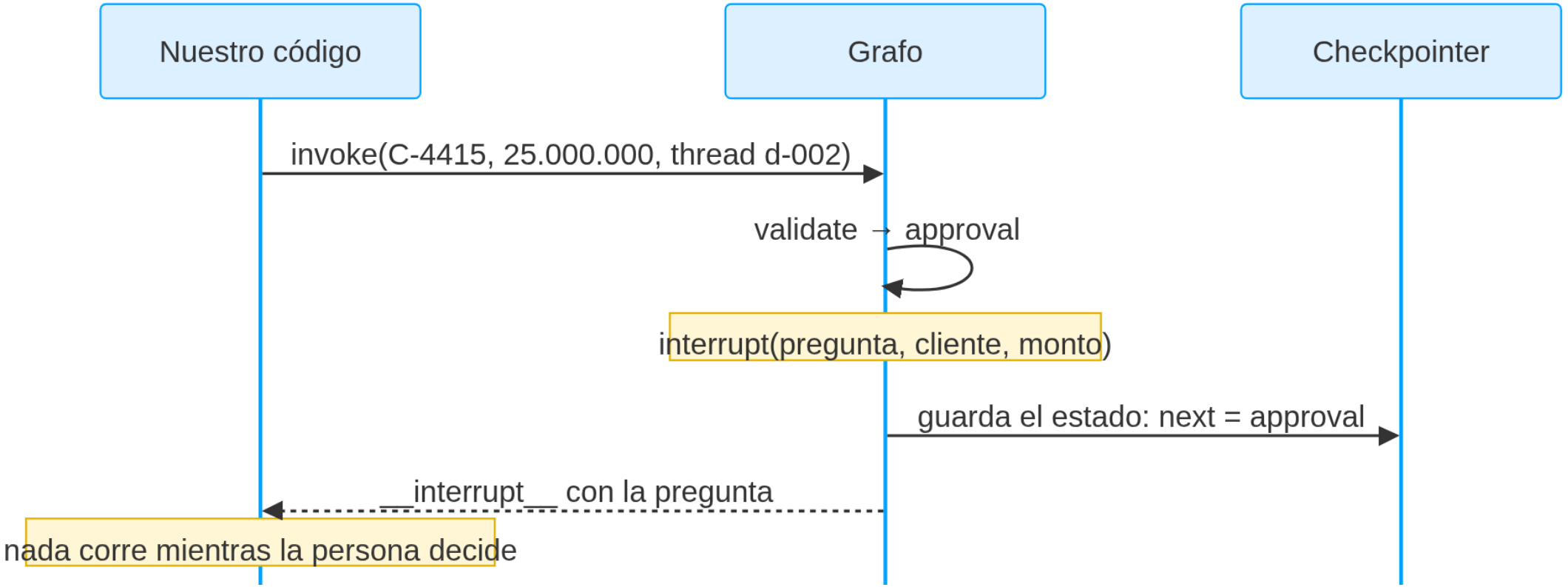
`interrupt` detiene el grafo y devuelve la pregunta a la aplicación.

```
from langgraph.types import Command, interrupt

def approval(state) -> Command[Literal["disburse", "reject"]]:
    decision = interrupt({
        "pregunta": "¿Apruebas este desembolso?",
        "cliente": state["customer"],
        "monto_cop": state["amount_cop"],
        "opciones": ["approve", "reject", "edit"],
    })
    if decision["type"] == "approve":
        return Command(goto="disburse")
    if decision["type"] == "edit":
        return Command(goto="disburse",
                       update={"amount_cop": decision["amount_cop"]})
    return Command(goto="reject", update={...})
```

La pausa, paso a paso

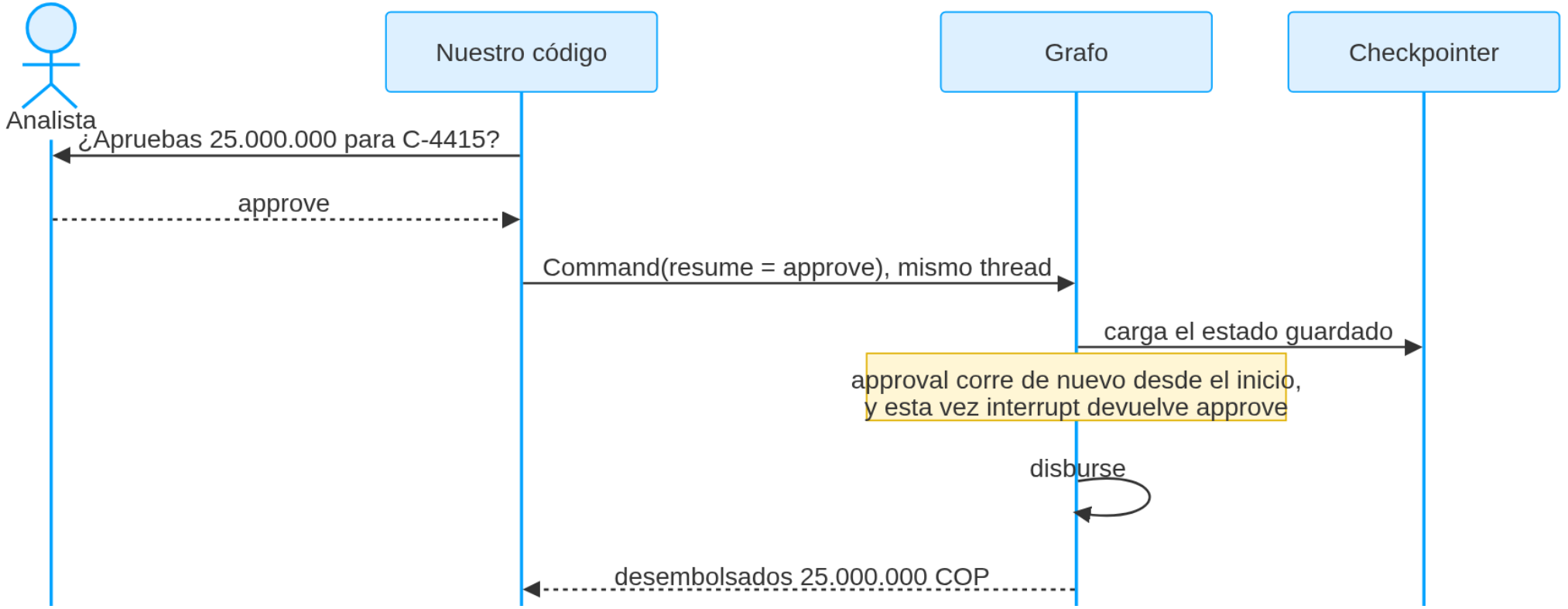
L20 · diagrama de secuencia



Pausar. `interrupt` detiene el grafo y el checkpointer guarda dónde quedó; el valor llega a la aplicación en `__interrupt__`.

La reanudación, paso a paso

L20 · diagrama de secuencia



Reanudar. `Command(resume=...)` con el mismo `thread_id` continúa: el nodo de aprobación se ejecuta otra vez y `interrupt` devuelve la decisión.

La pausa y lo que queda guardado

L20 · sección 3 · C-4415 pide 25.000.000 COP

Lectura	Valor
`__interrupt__`	¿Apruebas este desembolso? · C-4415 · 25.000.000
`state.next`	('approval',)
`state.values`	customer, amount_cop, status = validado
`thread_id`	d-002: la pausa sobrevive mientras exista el checkpointer

Reanudar con Command

L20 · sección 4 · sin modelo

Decisión	Command(resume=...)	Resultado
Aprobar	`{"type": "approve"}`	Desembolsados 25.000.000 COP
Rechazar	`{"type": "reject", "note": ...}`	Rechazado: falta la declaración de renta
Editar	`{"type": "edit", "amount_cop": 20_000_000}`	Desembolsados 20.000.000 COP

El nodo se ejecuta dos veces

Un detalle que cambia el diseño

Antes de interrupt

1. Al reanudar, el nodo vuelve a correr desde su primera línea.
2. Todo lo que esté antes de `interrupt` se ejecuta dos veces.
3. Por eso la acción (desembolsar, enviar) va en un nodo después de la aprobación.
4. Antes de `interrupt`, solo lecturas sin efectos.

Pontificia Universidad JAVERIANA Colombia

La aprobación en un agente

HumanInTheLoopMiddleware

Aprobar las herramientas de un agente

L20 · sección 5 · unas 4 llamadas

Middleware

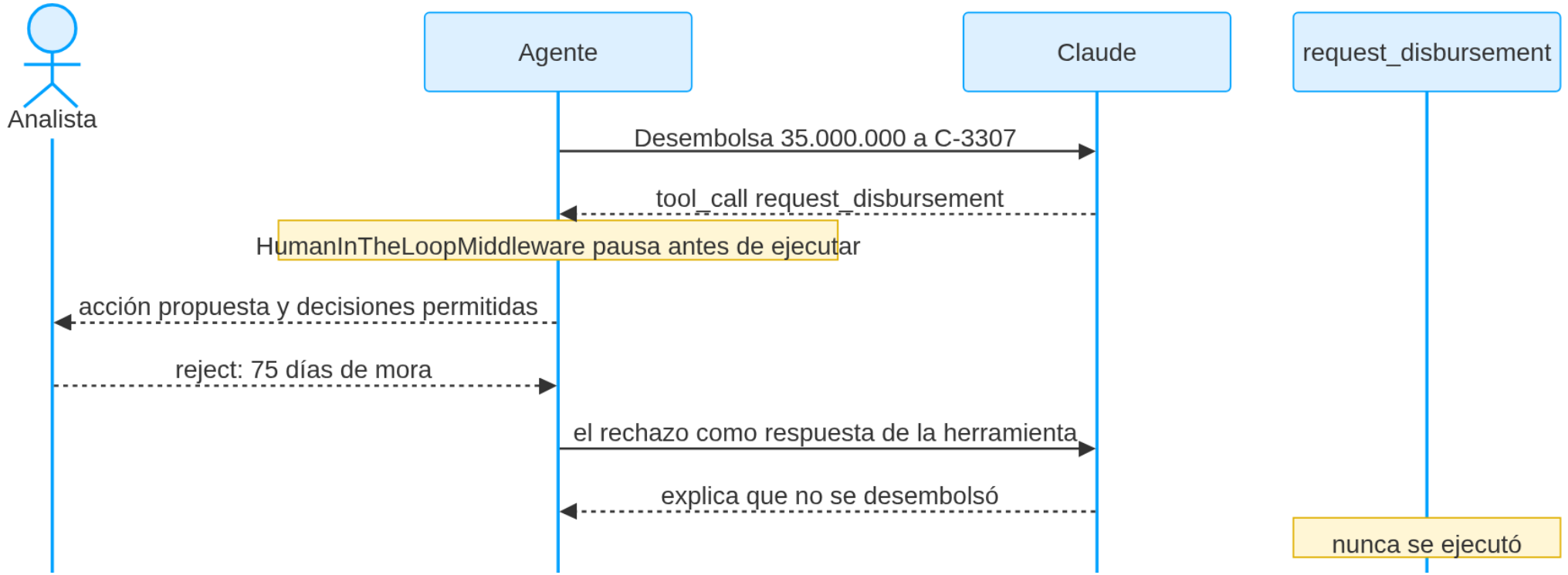
La herramienta no se ejecuta hasta que una persona decide.

```
from langchain.agents.middleware import HumanInTheLoopMiddleware

agent = create_agent(
    model, tools=[request_disbursement],
    middleware=[HumanInTheLoopMiddleware(
        interrupt_on={"request_disbursement": True})],
    checkpointer=InMemorySaver(),
)
paused = agent.invoke({"messages": [HumanMessage(
    "Desembolsa 25.000.000 COP al cliente C-4415.")]}, config)
# acción propuesta: request_disbursement(C-4415, 25000000)
agent.invoke(Command(resume={"decisions": [
    {"type": "approve"}]}), config)
```

Un rechazo con el middleware

L20 · diagrama de secuencia



La herramienta no corre. El middleware pausa antes de ejecutar; al rechazar, el modelo recibe el motivo y la herramienta nunca se ejecuta.

Qué cambia con la persona en el flujo

L20 · sección 6

El control en el código

1. Un prompt que dice «pide aprobación» puede ignorarse; un `interrupt` no.
2. Al rechazar, la herramienta nunca se ejecuta: se comprueba en los `ToolMessage`.
3. El estado de la pausa debe persistir: en producción, un checkpointer en base de datos.
4. La persona necesita contexto para decidir: cliente, monto, mora, política.

Dónde poner la aprobación

Opciones y lo que garantizan

Opción	Garantiza	Cuándo
Una regla en el prompt	Nada: el modelo puede ignorarla	Nunca como único control
`interrupt` en un nodo	El grafo no avanza sin la decisión	Flujos con pasos y límites conocidos
`HumanInTheLoopMiddleware`	La herramienta no corre sin la decisión	Agentes que eligen sus herramientas
Límites en código	Montos fuera de rango no llegan a la persona	Siempre, junto con la aprobación

Pontificia Universidad JAVERIANA Colombia

Laboratorio

L20-HumanInTheLoop

¿Qué haremos?

Laboratorio

Nuestro recorrido

1. Ejecutar el flujo sin persona: un desembolso pequeño se hace solo.
2. Pausar uno de 25.000.000 y leer lo que quedó guardado.
3. Reanudar aprobando, rechazando y editando el monto.
4. Poner la aprobación en la herramienta de un agente (unas 4 llamadas).

Las ideas que nos llevamos

Cierre

Criterios de construcción

1. Las acciones irreversibles necesitan un punto de control.
2. `interrupt` + checkpointer + `thread_id` = pausar y continuar.
3. La persona puede aprobar, rechazar o corregir.
4. Los límites de negocio viven en el código, no en el prompt.

Para seguir leyendo

Lecturas de la semana

Fuente	Qué revisar
Documentación de LangGraph	Interrupts y persistence
Documentación de LangChain	Human-in-the-loop middleware
Building Applications with AI Agents	Supervisión humana y confianza

Pontificia Universidad JAVERIANA Colombia

Próxima semana

Orchestrator-workers: dividir un ticket y delegar con `Send`.
Notebook: `L21-Orchestrator.ipynb`

Sergio Alberto Mora Pardo - H1 2027
GitHub · LinkedIn · Repositorio del curso